

ZESPÓŁ SZKÓŁ ELEKTRYCZNO-MECHANICZNYCH

Pracownia testowania i dokumentowania aplikacji

Testowanie aplikacji

Autor: Dawid Łazarz Klasa: 4P Prowadzący: mgr inż. Kamil Wojnarowski

NOWY SĄCZ 2026 r.

Spis treści

- Wstęp | 4
 - Testy jednostkowe | 5
 - 1.1 Implementacja programu | 5
 - 1.2 Implementacja testów | 5
 - 1.3 Git | 5
- Podsumowanie | 9
- Bibliografia | 10

Wstęp

Testowanie oprogramowania jest jednym z kluczowych elementów procesu wytwarzania aplikacji, którego głównym celem jest weryfikacja poprawności działania systemu oraz wykrycie ewentualnych błędów na jak najwcześniejszym etapie rozwoju.

Podstawowym zagadnieniem w inżynierii oprogramowania jest zapewnienie jakości (ang. Quality Assurance). Testowanie pozwala upewnić się, że kod spełnia wymagania funkcjonalne oraz biznesowe. W procesie testowym wyróżniamy różne poziomy testów, takie jak testy jednostkowe, integracyjne, systemowe oraz akceptacyjne. Wczesne wykrywanie usterek drastycznie redukuje koszty naprawy błędów oraz zwiększa bezpieczeństwo i stabilność tworzonych rozwiązań.

1. Testy jednostkowe

Testy jednostkowe (ang. Unit Tests) polegają na izolowanym sprawdzaniu najmniejszych fragmentów kodu, takich jak pojedyncze metody lub klasy, w celu weryfikacji ich prawidłowego działania. Wykonujemy je na bieżąco podczas pisania kodu (często zgodnie z metodologią Test-Driven Development - TDD).

Przykład opisowy: Wyobraźmy sobie kalkulator z funkcją dzielenia. Test jednostkowy podaje do metody dwie liczby (np. 10 i 2) i sprawdza, czy wynik wynosi dokładnie 5. Osobny test weryfikuje przypadek skrajny - przekazanie zera jako dzielnika i sprawdzanie, czy system poprawnie zgłosi błąd dzielenia przez zero.

Narzędzie xUnit: W projekcie wykorzystano xUnit - nowoczesny, otwarty framework testowy dla platformy .NET. Pozwala on na łatwe definiowanie przypadków testowych za pomocą adnotacji (np. [Fact]) oraz weryfikację wyników przy użyciu asercji (np. Assert.Equal, Assert.Throws).

1.1 Implementacja programu

W ramach zadania zaimplementowano w języku C# program realizujący algorytm sortowania bąbelkowego (Bubble Sort) wraz z walidacją danych wejściowych.

Kod programu (Program.cs):

```
C#
1. using System;
2.
3. public class Program
4. {
5.     public static void Main(string[] args)
6.     {
```

```
7.     int[] numbers = { 64, 34, 25, 12, 22, 11, 90 };
8.
9.     Console.WriteLine("Tablica przed sortowaniem:");
10.    PrintArray(numbers);
11.
12.    BubbleSort(numbers);
13.
14.    Console.WriteLine("Tablica po sortowaniu:");
15.    PrintArray(numbers);
16. }
17.
18. public static void BubbleSort(int[] array)
19. {
20.     // WARUNEK BEZPIECZEŃSTWA: Jeśli tablica jest null lub pusta, przerywamy
21.     if (array == null || array.Length == 0) return;
22.
23.     // WALIDACJA (Test negatywny): Ograniczenie rozmiaru tablicy do 1000 elementów
24.     if (array.Length > 1000)
25.     {
26.         throw new ArgumentException("Tablica jest zbyt duża do sortowania bąbelkowego!");
27.     }
28.
29.     int n = array.Length;
30.
31.     for (int i = 0; i < n - 1; i++)
32.     {
33.         for (int j = 0; j < n - i - 1; j++)
34.         {
35.             if (array[j] > array[j + 1])
36.             {
37.                 int temp = array[j];
38.                 array[j] = array[j + 1];
39.                 array[j + 1] = temp;
40.             }
41.         }
42.     }
43. }
44.
45. public static void PrintArray(int[] array)
46. {
47.     if (array == null) return;
48.     Console.WriteLine(string.Join(", ", array));
49. }
```

50. }

Opis działania kodu: Metoda BubbleSort przyjmuje jako parametr tablicę liczb całkowitych. Na początku weryfikuje warunki brzegowe: brak tablicy (null) lub pustą tablicę. Ponadto zawiera regułę biznesową ograniczającą wielkość tablicy do 1000 elementów - przekroczenie tego limitu skutkuje rzuceniem wyjątku ArgumentException. Sam algorytm wykorzystuje dwie zagnieżdżone pętle for, które porównują sąsiadujące elementy i w razie potrzeby zamieniają je miejscami za pomocą zmiennej pomocniczej temp.

1.2 Implementacja testów

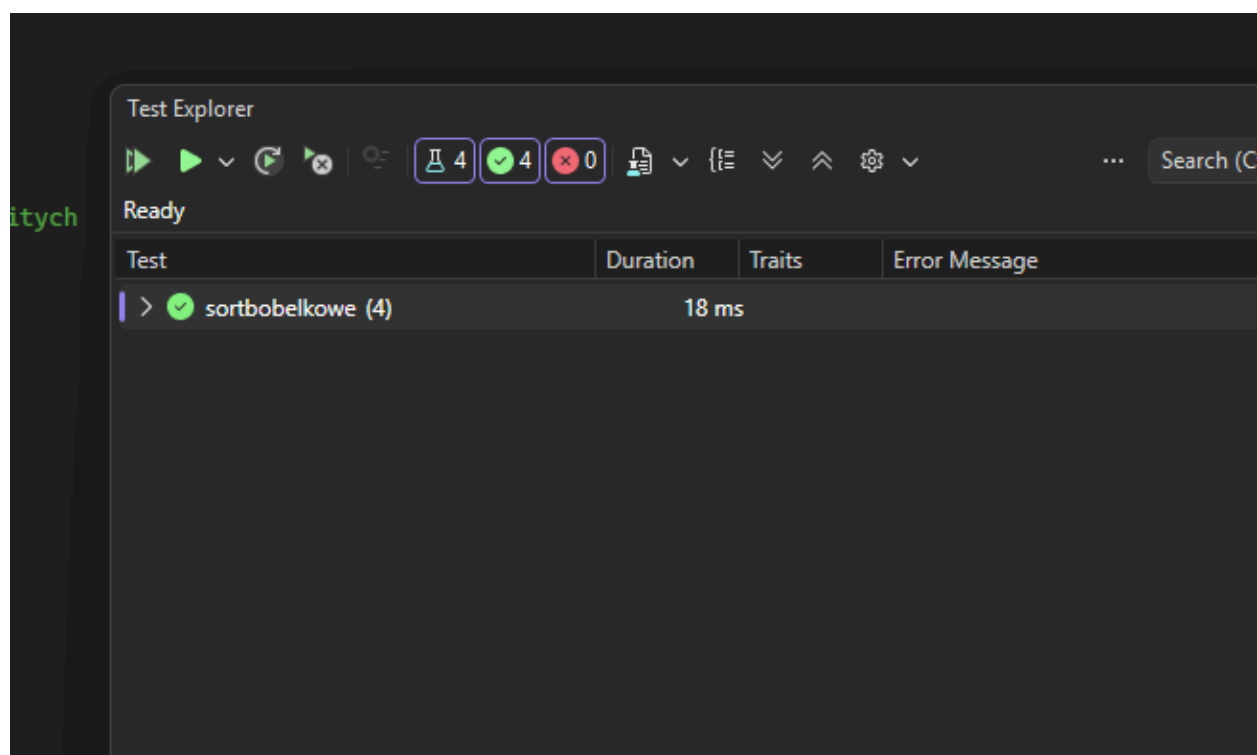
Zostały zaimplementowane 4 testy jednostkowe przy użyciu biblioteki xUnit (3 testy pozytywne oraz 1 test negatywny).

Kod testów (sortBobelkoweTest.cs):

```
C#
51. using System;
52. using Xunit;
53.
54. public class BubbleSortTests
55. {
56.     // --- TESTY POZYTYWNE ---
57.
58.     [Fact]
59.     public void Test_ValidArray_SortsCorrectly()
60.     {
61.         // Sprawdzamy czy metoda poprawnie sortuje zwykłą tablicę
62.         int[] input = { 3, 1, 2 };
63.         Program.BubbleSort(input);
64.         Assert.Equal(new[] { 1, 2, 3 }, input);
65.     }
66.
67.     [Fact]
68.     public void Test_NullArray_HandlesGracefully()
69.     {
70.         // Sprawdzamy czy program nie wywali się przy wartości null
71.         int[]? input = null;
72.         Program.BubbleSort(input);
73.         Assert.Null(input);
74.     }
75.
76.     [Fact]
```

```
77. public void Test_EmptyArray_HandlesGracefully()
78. {
79.     // Sprawdzamy czy program poprawnie obsługuje pustą tablicę
80.     int[] input = { };
81.     Program.BubbleSort(input);
82.     Assert.Empty(input);
83. }
84.
85. // --- TEST NEGATYWNY ---
86.
87. [Fact]
88. public void Test_TooLargeArray_ThrowsArgumentException()
89. {
90.     // Podajemy za dużą tablicę (ponad 1000 elementów)
91.     int[] tooLargeArray = new int[1001];
92.
93.     // Sprawdzamy czy algorytm wyrzuci oczekiwany błąd (ArgumentException)
94.     Assert.Throws<ArgumentException>(() => Program.BubbleSort(tooLargeArray));
95. }
96. }
```

Wyniki wykonania testów: Wszystkie testy zostały uruchomione w oknie Test Explorer środowiska Visual Studio i zakończyły się sukcesem (4/4 passed).



1.3 Git

W trakcie tworzenia programu oraz zestawu testów prowadzona była kontrola wersji przy użyciu systemu Git. Wykonano wymagane commity rejestrujące postęp prac nad kodem źródłowym i testami.

```

dawid@tuf UCRT64 ~/Desktop/projekt (main)
$ git log
commit 63c5ed82efb35a7c1da776dd1a655fc1bf2f2577 (HEAD -> main, origin/main)
Author: Slejerr <dawid15463@gmail.com>
Date: Thu Oct 8 20:47:06 2026 +0200

    zmiana czcionki w pdf'ie

commit 0349bea0e64d59182086cc1bda9a70d23e457d10
Author: Slejerr <dawid15463@gmail.com>
Date: Thu Oct 8 20:39:18 2026 +0200

    Zaktualizowano testy jednostkowe, zastąpienie starych testów nowymi

commit d4263479046c5dfa7b64c1b6ae66013bb1ad21ed
Author: Slejerr <dawid15463@gmail.com>
Date: Thu Oct 8 20:22:32 2026 +0200

    Ostatnie szlify, komentarze, dodanie negatywnego testu

commit 77fc76365aca3d43845f981fc9fc84bc86dc04b9
Author: Slejerr <dawid15463@gmail.com>
Date: Mon Oct 5 21:22:56 2026 +0200

    dodanie screena z testami

commit f91cad0f376e4d37e7de7b0f3aa3139128e84d40
Author: Slejerr <dawid15463@gmail.com>

dawid@tuf UCRT64 ~/Desktop/projekt (main)
$ git log --oneline
63c5ed8 (HEAD -> main, origin/main) zmiana czcionki w pdf'ie
0349bea Zaktualizowano testy jednostkowe, zastąpienie starych testów nowymi
d426347 Ostatnie szlify, komentarze, dodanie negatywnego testu
77fc763 dodanie screena z testami
f91cad0 Koniec i dodanie wszystkich plików z kodem i testem na nowo bo coś ZNOWU sie popsuło
ef62e26 implementacja programu/poprawa testów jednostkowych
b8ca5ee testy jednostkowe
9f581af dodanie png git log
7f9fd66 Dodanie szczegółowych komentarzy do kodu i funkcji pomocniczych
979431c Dodanie algorytmu sortowania babelkowego
246873f Implementacja funkcji pomocniczej Swap do zamiany elementów
2f38f32 Dodanie struktury programu i metody do wyświetlania tablicy
c589c7b (origin/testy, testy) opis zmian
15ed7a5 Dodanie zmiennej globalnej
b8e8740 Inicjalizacja repozytorium

```

Podsumowanie

W ramach przeprowadzonego zadania pomyślnie zaimplementowano algorytm sortowania bąbelkowego w języku C# oraz stworzono zestaw automatycznych testów jednostkowych w frameworku xUnit. Przeprowadzone testy (3 pozytywne oraz 1 negatywny) potwierdziły poprawność działania programu zarówno dla standardowych danych, przypadków brzegowych, jak i sytuacji zgłaszania błędów walidacji.

Bibliografia

1. Dokumentacja języka C# i .NET: <https://learn.microsoft.com/dotnet/csharp/>
2. Dokumentacja frameworka xUnit: <https://xunit.net/docs/getting-started/netcore/cmdline>

-
-
3. Dokumentacja systemu kontroli wersji Git: <https://git-scm.com/doc>